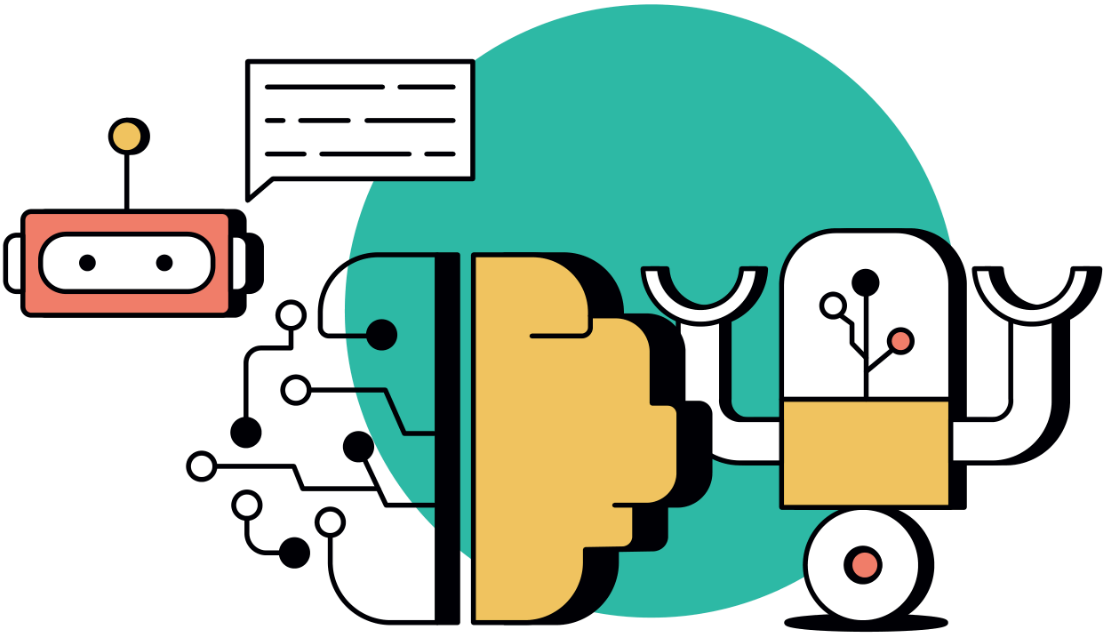
جزوه رشته هوش مصنوعی



استاد	محمد بهجت پور
درس	یادگیری ماشین
فصل	دسته‌بندی
مبحث	الگوریتم پرسپترون

به نام خدا

مسئله دسته‌بندی

D مجموعه داده → $D=\{(x_i,y_i)\}_{i=1}^{N}$

x بردار ویژگی → $x=\begin{bmatrix}x_1\\x_2\\\vdots\\x_n\end{bmatrix}$ $x\in\mathbb{R}^n$

بردار x، n بعدی است.

بردار x، n مؤلفه دارد

y_i: برچسب داده iام → $y_i\in\{0,1\}$

$0\rightarrow -1$، $1\rightarrow +1$

دسته‌بندی دوکلاسه یا Binary Classification

→ One vs Rest

One v's One

قطر اصلی ماتریس A

$A=\begin{bmatrix}1&-1\\3&2\end{bmatrix}_{2\times2}\rightarrow A^T=\begin{bmatrix}1&3\\-1&2\end{bmatrix}_{2\times2}$

$a_{ij}\rightarrow a_{ji}$

Transpose

ترانهاده کردن ماتریس/بردار

قرینه کردن ماتریس نسبت به قطر اصلی. مثال:

سطری ← ستونی / ستونی ← سطری. مثال:

$v=\begin{bmatrix}v_1\\v_2\\v_3\end{bmatrix}\rightarrow v^T=[v_1\ \ v_2\ \ v_3]$

ایده اصلی پرسپترون: ابتدا ویژگی‌ها را در وزن‌های مدل ضرب می‌کنیم و حاصل را با بایاس (Bias) جمع می‌کنیم:

$$z_i=w^T\cdot x_i+b$$

نماد ترانهاده

بایاس

بردار وزن‌ها

نماد ضرب داخلی دو بردار

بردار ویژگی نمونه iام

$y=ax+b$

شیب خط

عرض از مبدأ

$w=\begin{bmatrix}w_1\\w_2\\\vdots\\w_n\end{bmatrix}\rightarrow w^T=[w_1\ \ w_2\ \dots\ w_n]$

وسپس اگر حاصل مثبت بود، مدل یک کلاس را پیش‌بینی می‌کند و اگر صفر یا منفی بود، کلاس دیگر را:

$$h(z)=\begin{cases} z_i\le 0 \rightarrow \text{کلاس } -1\\ 0<z_i \rightarrow \text{کلاس } +1\end{cases}$$

$w^Tx+b=0$ → یک خط است

⟸ مرز تصمیم مدل خطی است

$$y_i=h(w^T\cdot x_i+b)$$

کلاس پیش‌بینی شده برای نمونه iام.

تفسیر معانی وزن‌ها: مثلاً در

$z=3x_1-2x_2+4$

w_1، w_2، b

پس وزن مثبت یعنی افزایش ویژگی متناظر با آن، باید z را افزایش دهد.

و وزن منفی یعنی افزایش مقدار ویژگی متناظر با آن، باید مقدار z را کاهش دهد.

و بزرگی وزن نشان می‌دهد آن ویژگی چه مقدار روی مقدار خطی z اثر می‌گذارد. (البته این تفسیر به مقیاس ویژگی‌ها نیز وابسته است)

$y=ax+b\rightarrow y=ax\rightarrow(0,0)\in d$ → خط d از مبدأ گذر است.

کاربرد Bias در مدل: اگر Bias نداشتیم، مدل به صورت زیر می‌بود:

$z=w^T\cdot x$

$\rightarrow z=w_1x_1+w_2x_2$ → در حالت دوبعدی

این خط لزوماً از مبدأ مختصات عبور می‌کند. اما با افزودن b به z، خط می‌تواند جابجا شود و لازم نیست از مبدأ عبور کند.

در نتیجه Bias آزادی بیشتری برای تعیین مرز تصمیم به مدل می‌دهد.

مرز تصمیم: به طور کلی مرز تصمیم جایی است که مدل بین دو کلاس مردد است. در پرسپترون این مرز زمانی اتفاق می‌افتد که $z=۰$ شود. یعنی $w^Tx+b=۰$.

مثلاً اگر داشته باشیم $۲x_1+۳x_2-۶=۰$، مرز تصمیم مدل همین خط است، و اگر $۲x_1+۳x_2-۶<۰$ باشه نمونه به کلاس $-$ و اگر $۲x_1+۳x_2-۶>۰$ باشه، نمونه به کلاس $+$ تعلق دارد.

پس پرسپترون، فضای ویژگی (Feature Space) را به ۲ بخش تقسیم می‌کنه.

در دو بعد مرز تصمیم خط است. در سه بعد صفحه است و در ابعاد بالاتر Hyper Plain یا ابرصفحه است.

مثال: فرض کنید پرسپترون روبرو را داریم، نمونه $x=(۸,۳,-۲)$ به کدام دسته تعلق دارد؟ $w=\begin{bmatrix}۲\\-۱\\۴\end{bmatrix}$ $b=-۳$

$$z=w^T.x+b \longrightarrow [۲\ -۱\ ۴].[۸\ ۳\ -۲]-۳ = ۱۶-۳-۸-۳=۲$$

$$y=h(z)$$

$\hookrightarrow h(z)=h(۲)=+۱ \rightarrow$ پس این نمونه به کلاس +۱ تعلق دارد.

فرآیند یادگیری مدل: در یادگیری ماشین ما وزن‌ها را نمی‌دانیم، ولی در مرحله نخست مقادیر آنها را به صورت تصادفی تعیین می‌کنیم.

مثلاً. $w=\begin{bmatrix}-۳٫۵\\۴٫۹\end{bmatrix}$ و $b=۹٫۱$ یا $w=\begin{bmatrix}۱٫۴\\۳٫۵\end{bmatrix}$ و $b=-۰٫۲$

سپس داده‌های آموزش را یکی یکی به مدل می‌دهیم. اگر مدل یک نمونه را درست دسته‌بندی کرد کاری نمی‌کنیم، اما اگر مدل یک نمونه را نادرست طبقه‌بندی کرده بود وزن‌ها را تغییر می‌دهیم.

$$w \leftarrow w+\overset{n}{\alpha}\, y_i x_i \qquad b \leftarrow b+\alpha y_i \qquad y_i \in \{-۱,+۱\}$$

$\alpha \rightarrow$ نرخ یادگیری

↳ هایپرپارامتر ← برای شروع می‌توانیم $\alpha=۰٫۱$ یا $\alpha=۰٫۰۱$ یا ساده‌تر $\alpha=۱$ بگذاریم.

مثال: فرض کنید داده‌هایی مطابق جدول زیر به ما داده شده است. وزن‌های مدل را از $w=(۰,۰)$ و $b=۰$ یک مرحله به‌روزرسانی کنید؟

x_1	x_2	y
۲	۱	+۱
۱	۲	−۱

$\longrightarrow w \leftarrow \begin{bmatrix}۰\\۰\end{bmatrix}+۰٫۱\times۱\times\begin{bmatrix}۲\\۱\end{bmatrix} \rightarrow w=\begin{bmatrix}۰٫۲\\۰٫۱\end{bmatrix}$ Epoch 1

$b \leftarrow ۰+۰٫۱\times+۱ \rightarrow b=۰٫۱$

$w \leftarrow \begin{bmatrix}۰٫۲\\۰٫۱\end{bmatrix}+۰٫۱\times(-۱)\times\begin{bmatrix}۱\\۲\end{bmatrix} \rightarrow \begin{bmatrix}۰٫۱\\-۰٫۱\end{bmatrix} \longrightarrow$ وزن‌های نهایی

Epoch 2

$b \leftarrow ۰٫۱+۰٫۱\times(-۱) \rightarrow b=۰ \rightarrow$ بایاس نهایی

صورت الگوریتم پرسپترون:

ورودی: ۱- داده های آموزشی: x_i ۲- برچسب ها: y_i
۳- نرخ یادگیری (Learning Rate): α ۴- تعداد epoch ها

مراحل: ۱- مقداردهی اولیه وزن ها ۲- مقدار دهی اولیه Bias

۳- برای هر نمونه داده آموزشی:
۳,۱- محاسبه $z = w^T x_i + b$
۳,۲- به روزرسانی وزن ها اگر $y_i \times h(z) \leqslant 0$

۴- بررسی همه نمونه ها و در صورت لزوم تکرار مرحله ۳ برای چند epoch.
+ اگر در یک epoch همه نمونه ها به درستی دسته بندی شده بودند، فرآیند آموزشی را متوقف می کنیم.

epoch چیست؟ به هربار بررسی تمامی داده های آموزشی، یک epoch گفته می شود. مثلاً اگر ۱۰۰ داده آموزشی داشته باشیم، epoch ها به صورت روبرو خواهند بود:

Epoch 1: هر ۱۰۰ نمونه یک بار بررسی می شوند.
Epoch 2: هر ۱۰۰ نمونه دوباره بررسی می شوند.
Epoch 3: . . . .
. . . .
. . . .

هدف این است تا در نهایت به نقطه ای برسیم که همه نمونه های آموزشی به درستی طبقه بندی شوند.

آیا پرسپترون همیشه همگرا می شود؟ خیر - اگر داده ها به صورت خطی جداپذیر باشند این الگوریتم پس از تعداد محدودی آپدیت به یک جداکننده خطی می رسه. اما اگر داده ها به صورت خطی جداپذیر نباشند، پرسپترون دائماً وزن ها را تغییر میده اما نمی تواند همه داده ها را درست طبقه بندی کنه.

این موضوع به Perceptron Convergence Theorem معروف است.

معنی جداپذیری خطی: اگر بتوانیم خط/صفحه/ابرصفحه ای بیابیم که دو کلاس مثبت و منفی را از هم جدا کنه، داده ها جداپذیر خطی هستند.

مثال XOR: داده های جدول روبرو را نمی توان به صورت خطی جدا کرد. بنابراین این داده ها را نمی توان با یک پرسپترون ساده دسته بندی کرد.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

در پرسپترون کلاسیک از تابع فعالسازی پله ای استفاده می شود. اما در شبکه عصبی مدرن از توابعی مثل Sigmoid، Softmax، Tanh، ReLU و ... استفاده می شود.